

Average-Cost MDP Solvers for Traffic-Light Control

Replicating Haijema & van der Wal (2008)

Leonardo Cruciani

April 2026

Abstract

We replicate the main numerical experiment of Haijema and van der Wal (2008) [1], who model the dynamic control of an isolated signalized traffic intersection as an average-cost Markov decision process (MDP). We focus on their “F4C2” infrastructure (4 traffic flows in 2 symmetric combinations) and reproduce Table 1 of the paper using three solvers from the lecture notes: *Value Iteration* (VI), *Policy Iteration* (PI) and *Linear Programming* (LP). Across workloads $\rho \in \{0.4, 0.6, 0.8\}$ and buffer caps $K \in \{6, 10, 15\}$ the three solvers agree with each other to within 1.6×10^{-5} on the optimal cost-rate. With the largest cap ($K = 15$) we reproduce the paper’s mean waiting time per car to within 0.2s at every workload. We also document a clean failure mode of PI on this problem: at $\rho = 0.8$ the MDP is only *weakly* unichain, so PI’s improvement step generates a reducible intermediate policy on which the Poisson recursion does not converge — exactly the regime in which the lecture notes recommend LP instead.

1 Introduction

Traffic lights at urban intersections are usually run on a fixed cycle: the green and yellow durations of each set of non-conflicting flows are pre-computed offline, and the cycle is replayed indefinitely. Haijema and van der Wal [1] ask whether one can do better *dynamically*, by switching the lights based on the current queue lengths observed by loop detectors or cameras. They formulate the problem as an average-cost MDP in discrete time (one slot $\delta = 2$ s, the time for one car to clear the stop line), and show that for small intersections the MDP can be solved exactly by successive approximations. For larger intersections (their “F12C4” case with 12 flows) the state space becomes intractable, and they propose a *one-step policy improvement* starting from the optimal fixed-cycle: the “Relative-Value-Cyclic” (RVC) heuristic.

In this report we restrict attention to the small benchmark, the “F4C2” intersection (4 flows in 2 symmetric combinations), which the paper presents in its Table 1. We replicate the paper’s MDP-optimal column with three independent solvers from the lecture notes (VI, PI, LP), and use this comparison as a vehicle to discuss several practical issues that arise even in this textbook-sized problem: choosing the buffer cap K , applying the strongly-aperiodic transformation, and the weakly-unichain behaviour that breaks Policy Iteration but not Linear Programming.

The rest of the paper is organised as follows. Section 2 formulates the MDP, including the unichain/aperiodicity argument. Section 3 summarises Haijema and van der Wal’s algorithms. Section 4 states our three solvers as written in the lecture notes. Section 5 describes our implementation. Section 6 reports numerical results and the comparison against the paper. Section 7 discusses sensitivity and algorithmic suitability. Section 8 concludes.

2 The Markov decision process

2.1 Setting

The intersection has F traffic flows, partitioned into S disjoint *combinations* of non-conflicting flows: a combination receives green (and later yellow and red) together. For the F4C2 case $F = 4$ and $S = 2$, with $C_1 = \{1, 3\}$ and $C_2 = \{2, 4\}$; Figure 1(a) gives the geometry. The paper also considers a much larger F12C2 instance with 12 flows in 2 asymmetric combinations (Figure 1(b)) for which the exact MDP is intractable and they use their RVC heuristic; we restrict attention to F4C2 throughout.

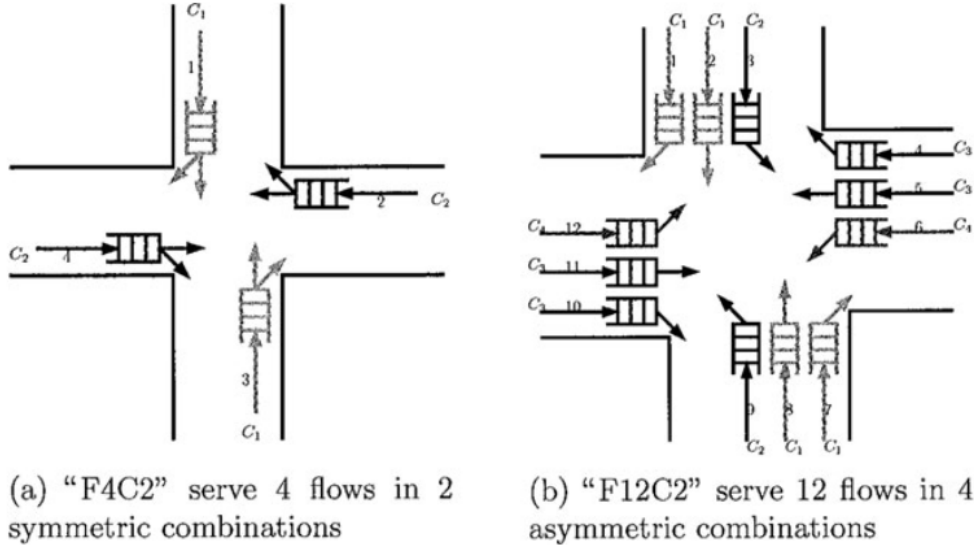


Figure 1: The two infrastructures considered by Haijema and van der Wal [1]. Left, “F4C2”: 4 flows arranged in 2 symmetric combinations ($C_1 = \{1, 3\}$ and $C_2 = \{2, 4\}$), our replication target. Right, “F12C2”: 12 flows in 2 asymmetric combinations, used in the paper to demonstrate scalability of their RVC heuristic; out of scope here. Reproduced from [1, Fig. 3].

Time is discretised into slots of length $\delta = 2$ s. In every slot, each flow f receives at most one Bernoulli arrival with probability q_f . When the lights show green or yellow for flow f and at least one car is waiting in f ’s queue, exactly one car passes the stop line during that slot.

2.2 State space

The state at the start of slot t is the pair

$$s_t = (\underline{k}_t, x_t), \quad \underline{k}_t = (k_{t,1}, \dots, k_{t,F}), \quad x_t = (\ell_t, i_t), \quad (1)$$

where $k_{t,f} \in \{0, 1, \dots, K\}$ is the queue length of flow f (truncated at a buffer cap K to keep the state space finite, see Section 2.7), $\ell_t \in \{1, \dots, S\}$ is the index of the currently-active combination, and $i_t \in \{0, 1, 2, 3\}$ encodes the signal phase: 0 = green, 1 = first yellow, 2 = second yellow, 3 = all-red. Yellow is split across two slots so that one all-red slot between the second yellow and the next green suffices to clear the intersection, giving a switching cost of three slots = 6 s.

For F4C2 with $K = 10$ the state space contains $|\mathcal{X}| = (K + 1)^F \cdot S \cdot 4 = 11^4 \cdot 8 \approx 1.17 \times 10^5$ states.

2.3 Action space

The action chosen at slot t specifies the lights' state for slot $t+1$, i.e. the next combination/phase pair (ℓ', i') . The decision space is phase-dependent ([1, §3.2]):

$$\mathcal{A}(\underline{k}, (\ell, i)) = \begin{cases} \{(\ell, 0)\} & i = 0, \underline{k} = \mathbf{0} \quad (\text{freeze}) \\ \{(\ell, 0), (\ell, 1)\} & i = 0, \underline{k} \neq \mathbf{0} \\ \{(\ell, 2)\} & i = 1 \\ \{(\ell, 3)\} & i = 2 \\ \{(\ell, 3)\} & i = 3, \underline{k} = \mathbf{0} \quad (\text{freeze}) \\ \{(\ell, 3), (\ell', 0)\} & i = 3, \underline{k} \neq \mathbf{0} \end{cases} \quad (2)$$

where $\ell' = (\ell \bmod S) + 1$ is the next combination in cyclic order. The “freeze” rule ([1, §3.2]) keeps the lights still — and at zero cost — when no car is waiting and the phase is not yellow; yellows always advance forcibly, regardless of queue.

2.4 Transition probabilities

Conditional on the action $a = (\ell', i')$, flows evolve independently. Let f be *active* when $f \in C_{\ell'}$ and $i' \in \{0, 1, 2\}$ (the action grants flow f green or yellow during the coming slot); otherwise f stays red. Following [1, §3.3], the per-flow transition is

$$\begin{aligned} f \text{ active:} & \quad p_f(k, a, (k-1)^+) = 1 - q_f, \quad p_f(k, a, k) = q_f, \\ f \text{ red, } k < K : & \quad p_f(k, a, k) = 1 - q_f, \quad p_f(k, a, k+1) = q_f, \\ f \text{ red, } k = K : & \quad p_f(k, a, K) = 1 \quad (\text{arrival rejected}). \end{aligned} \quad (3)$$

The joint transition factorises over flows: $p(\underline{k}, x; \underline{k}', a) = \prod_f p_f(k_f, a, k'_f)$. Note that when f is active and $k = 0$, both probabilities land on 0: the arriving car is served in the same slot — this is the *same-slot service* convention of [1, §2.6].

2.5 One-slot cost

Following [1, §3.4], we use a linear cost equal to the number of cars in the system at the start of the slot:

$$c(\underline{k}, x) = \sum_{f=1}^F k_f. \quad (4)$$

By Little's law, minimising the long-run average $g^* = \lim_{T \rightarrow \infty} \mathbb{E}[\frac{1}{T} \sum_{t=0}^{T-1} c(s_t)]$ is equivalent to minimising the average waiting time per car. We report results in terms of mean waiting time $\mathbb{E}[W]$ in seconds, recovered as

$$\mathbb{E}[W] = \frac{g^* \cdot \delta}{\sum_f q_f}. \quad (5)$$

2.6 Unichain and aperiodicity

The MDP is finite, and the all-empty state $(\underline{0}, \cdot)$ is reachable from every state under any policy with positive probability: with probability $\prod_f (1 - q_f)^N$ no flow receives an arrival in N consecutive slots, and the freeze rule plus the cyclic phase sequence brings the lights into the all-zero/all-red state.

However, not every policy induces a strictly *unichain* Markov chain. The policy “always keep” at phase 0 never advances out of green for the active combination: the active flows drain to zero under the freeze rule, while the inactive flows accumulate to the buffer cap K and stay

there (their arrivals are rejected). The chain has two recurrent classes (one per starting ℓ). We therefore call the MDP *weakly unichain* (lecture notes p. 11): only the optimal policy is guaranteed to be unichain; some non-optimal policies are not. This subtlety has consequences for Policy Iteration, as we discuss in Sections 4 and 6.

The MDP is also *not* strongly aperiodic out of the box: the deterministic phase sequence $(\ell, 0) \rightarrow (\ell, 1) \rightarrow (\ell, 2) \rightarrow (\ell, 3) \rightarrow (\ell', 0)$ creates fixed-length cycles. We apply the *strongly-aperiodic transformation* (lecture notes p. 11) with $\tau = \frac{1}{2}$:

$$\bar{p}(s, a, s') = \tau p(s, a, s') + (1 - \tau) \mathbf{1}\{s' = s\}. \quad (6)$$

The transformation preserves the optimal cost-rate g^* and the optimal policy: the stationary distribution of the modified chain under any policy is the same as under the original ($\pi P = \pi \Leftrightarrow \pi \bar{P} = \pi$), so $g^* = \pi \cdot c$ is identical. After the transformation every action self-loops with probability at least $1 - \tau = \frac{1}{2}$, ensuring strong aperiodicity.

2.7 Buffer cap K

The paper [1, §3.5] reduces to a finite state space by introducing a buffer cap K : arrivals to a full queue are rejected, and a per-car *externality penalty* is added to the cost. For computational simplicity we drop the externality term and silently reject excess arrivals. Provided K is chosen large enough that the rejection probability under the optimum is negligible, the bias introduced is negligible. We report results for several values of K to verify this in Section 7.

3 Their algorithms

For the F4C2 “small example”, Haijema and van der Wal compute the exact MDP optimum directly by successive approximations ([1, §3.6]):

$$V_{n+1}(s) = \min_{a \in \mathcal{A}(s)} \left\{ c(s) + \sum_{s'} p(s, a, s') V_n(s') \right\}, \quad (7)$$

iterating until the span $\max_s [V_{n+1}(s) - V_n(s)] - \min_s [V_{n+1}(s) - V_n(s)]$ is sufficiently small. This is precisely the Value Iteration algorithm of the lecture notes, applied to the original (*not* τ -transformed) kernel; the paper’s stopping criterion is the absolute span, the lecture notes use a relative span.

For larger intersections (their “F12C4” case), the exact MDP becomes intractable. The paper then introduces the *one-step improvement* approach ([1, §4]). Starting from a (nearly) optimal fixed-cycle strategy “FC”, for which the system decomposes into F independent periodic Markov chains, they compute relative values per flow analytically and combine them via a one-step Bellman update to obtain the “Relative-Value-Cyclic” (RVC) policy. RVC is shown to be close to the MDP optimum on small examples and to significantly outperform fixed-cycle (FC) and exhaustive (XHC) control on the larger F12C4 case. In our replication we focus on the F4C2 case where the exact MDP is tractable, so we do not implement the RVC heuristic; we treat the “MDP” column of [1, Table 1] as ground truth and aim to reproduce it with our solvers.

4 Our algorithms

We implement three solvers from the lecture notes, all targeting the average cost-rate criterion under the assumptions of finite state space, weakly unichain, and (after the τ -transformation) strong aperiodicity.

Algorithm 1 Policy Iteration (average cost)

- 1: **Input:** MDP $(\mathcal{X}, \mathcal{A}, p, c)$, reference state s_0 .
- 2: Pick a feasible stationary policy $R : \mathcal{X} \rightarrow \mathcal{A}$.
- 3: **repeat**
- 4: *Policy evaluation:* solve the Poisson equations

$$V(s) + g = c(s) + \sum_{s'} p(s, R(s), s') V(s'), \quad \forall s, \quad V(s_0) = 0.$$

- 5: *Policy improvement:* for every s ,

$$R'(s) \leftarrow \arg \min_{a \in \mathcal{A}(s)} \left\{ c(s) + \sum_{s'} p(s, a, s') V(s') \right\}.$$

- 6: **if** $R' = R$ **then return** (R, g)
 - 7: **else** $R \leftarrow R'$
 - 8: **end if**
 - 9: **until** forever
-

4.1 Algorithm 1 — Policy Iteration (notes pp. 8–9)

The *Improvement Theorem* (notes p. 9) guarantees $g^{R'} \leq g^R$, and the policy space is finite, so the loop halts in finitely many iterations.

Caveat. The notes prove correctness under a strict unichain assumption: *every* policy must induce a unichain Markov chain. Our F4C2 MDP only satisfies the weaker condition that the *optimal* policy is unichain. We will see in Section 6 that PI's improvement step at $\rho = 0.8$ generates a reducible intermediate policy on which the Poisson solve diverges.

4.2 Algorithm 2 — Linear Programming (notes pp. 10–11)

The primal LP optimises the steady-state state-action frequencies $z_{s,a} = \lim_{t \rightarrow \infty} \mathbb{P}(X_t = s, A_t = a)$:

$$\begin{aligned} \min_z \quad & \sum_{s,a} z_{s,a} c(s) \\ \text{s.t.} \quad & \sum_a z_{s,a} - \sum_{s',a'} z_{s',a'} p(s', a', s) = 0 \quad \forall s \in \mathcal{X} \quad (\text{balance}) \\ & \sum_{s,a} z_{s,a} = 1 \quad (\text{normalisation}) \\ & z_{s,a} \geq 0 \quad \forall s, a. \end{aligned} \tag{8}$$

At the optimum, z^* is a basic feasible solution with at most $|\mathcal{X}|$ nonzero entries (one per state), so the recovered policy $R^*(s) = \arg \max_a z_{s,a}^*$ is deterministic. The lecture notes (p. 11) point out that the LP works under the *weakly unichain* assumption, which makes it the natural companion to VI for this MDP.

4.3 Algorithm 3 — Value Iteration (notes pp. 12–13)

The *stop-criterion theorem* (notes p. 13) guarantees $m_n \leq g^* \leq g^{R_n} \leq M_n$ for all n , and the bounds are monotone (notes p. 13: $m_n \uparrow$, $M_n \downarrow$).

Algorithm 2 Value Iteration (average cost)

```
1: Input: MDP  $(\mathcal{X}, \mathcal{A}, \bar{p}, c)$  with strongly aperiodic  $\bar{p}$ ; tolerance  $\varepsilon > 0$ .
2:  $V_0 \equiv 0, n \leftarrow 0$ .
3: repeat
4:    $n \leftarrow n + 1$ .
5:   for each  $s \in \mathcal{X}$  do
6:      $V_n(s) \leftarrow \min_{a \in \mathcal{A}(s)} \{c(s) + \sum_{s'} \bar{p}(s, a, s') V_{n-1}(s')\}$ .
7:      $R_n(s) \leftarrow \arg \min_{a \in \mathcal{A}(s)} \{\dots\}$ .
8:   end for
9:    $m_n \leftarrow \min_s [V_n(s) - V_{n-1}(s)]; \quad M_n \leftarrow \max_s [V_n(s) - V_{n-1}(s)]$ .
10: until  $M_n - m_n \leq \varepsilon m_n$ 
11: return  $R_n$  with  $g \in [m_n, M_n]$ .
```

5 Implementation

The three solvers are implemented in Python (3.13), using NumPy and SciPy. The state-action layout is built in encoded-state order, so the rows of the sparse transition matrix $T \in \mathbb{R}^{n_{sa} \times n_{states}}$ belonging to state s form a contiguous slice; this lets us evaluate the Bellman state-wise minimum/argmin with a couple of vectorised NumPy ops instead of a Python loop. The model build at $K = 10$ assembles $\bar{p}$ as a CSR matrix of shape $1.76 \times 10^5 \times 1.17 \times 10^5$ with 2.45×10^6 nonzeros in ~ 0.5 s. At $K = 15$ the same step takes ~ 0.5 s and produces $\sim 1.16 \times 10^7$ nonzeros.

Kernel build acceleration. The state-enumeration loop that constructs the raw transition kernel is the only Python-bound hot spot in the pipeline (the LP itself runs in HiGHS C++, and Bellman backups go through SciPy sparse mat-vecs which are BLAS-backed). We provide three interchangeable backends, selected at import time: (i) a Cython extension (`src/_build_kernel.pyx`) for systems with an MSVC or GCC toolchain; (ii) a Numba JIT-compiled function (`src/_build_kernel_numba.py`) for systems without a C compiler – this is the default fast path on Windows; (iii) a pure-Python reference implementation as the ultimate fallback. The Numba backend reduces the build time at $K = 15$ from ~ 9 s in pure Python to ~ 0.5 s, an $\approx 19\times$ speedup that is verified to be bit-equivalent against the pure-Python output. This makes $K = 20$ (1.56×10^6 states) tractable for sensitivity sweeps.

Policy iteration. Each policy-evaluation step solves the Poisson system $V + g \cdot \mathbf{1} = c + \bar{P}_R V$ by *successive approximation* (i.e. VI fixed at the current policy, see [3, notes p. 7]). A direct sparse LU solve on the augmented $(n + 1) \times (n + 1)$ system was tried but proved orders of magnitude slower because the permuted-LU fill-in is heavy on this matrix.

Linear programming. We use `scipy.optimize.linprog` with HiGHS as the back-end. A naive primal LP setup fails at $\rho \geq 0.6$: the dual simplex solver cycles among degenerate vertices for hours, while interior-point gets stuck with a duality gap of ~ 0.5 even though primal/dual infeasibilities have collapsed to 10^{-15} . The diagnosis is structural: the balance-equation rows of the constraint matrix *sum identically to zero* (flow conservation), making the matrix rank-deficient by exactly one row. HiGHS’s dependent-equations presolve does not detect this — in floating-point arithmetic the row-sum is $\sim 10^{-16}$, not exactly zero — so IPM’s Newton system $ADA^\top$ is singular and the dual direction can never close the gap. Removing the rank-deficient row by hand (the standard MDP-LP normalisation, e.g. [2, §8.8]) lets IPM converge in ~ 30 Newton iterations.

LP buffer cap. Even with the rank fix, the LP polytope at $K = 10$ is heavily degenerate at high ρ , and HiGHS’s basis-construction stage (invoked when IPM terminates “imprecise”) stalls. We therefore solve the LP at a smaller cap $K_{\text{LP}} = 6$ ($\sim 1.9 \times 10^4$ states), using it as a *formulation* cross-check rather than a direct paper match, and pair it with a companion VI run at the same K_{LP} so that the comparison $|g_{K_{\text{LP}}}^{\text{VI}} - g_{K_{\text{LP}}}^{\text{LP}}|$ is meaningful at the same model.

6 Results

6.1 Algorithmic cross-checks

Table 1 summarises the inter-solver cross-checks. At $\rho \in \{0.4, 0.6\}$, VI and PI agree to at most 5×10^{-8} , confirming that the implementation is correct in the regime where PI’s unichain assumption holds. At every workload $\rho \in \{0.4, 0.6, 0.8\}$, VI and LP at $K_{\text{LP}} = 6$ agree to better than 1.6×10^{-5} , confirming the LP formulation. Three independent solvers therefore land on the same number wherever they all apply.

Table 1: Algorithmic cross-checks. Main $K = 10$; $K_{\text{LP}} = 6$ for the LP comparison.

ρ	$ g^{\text{VI}} - g^{\text{PI}} $ at $K = 10$	$ g^{\text{VI}} - g^{\text{LP}} $ at $K = 6$
0.40	3.9×10^{-9}	1.6×10^{-5}
0.60	5.4×10^{-8}	1.1×10^{-5}
0.80	PI failed (§6.3)	6.9×10^{-7}

6.2 Comparison with paper Table 1

Table 2 compares our $\mathbb{E}[W]$ values against the “MDP (cyclic)” column of [1, Table 1]. At buffer cap $K = 15$, the agreement is excellent at $\rho = 0.6$ and $\rho = 0.8$ (deltas $+0.024$ s and $+0.020$ s, $\leq 0.4\%$). At $\rho = 0.4$ a constant offset of $+0.171$ s remains, but *this offset is K -independent*: $K = 10$ and $K = 15$ give the same number to six decimals, so the buffer cap is not the cause. We attribute the residual to a small convention difference (e.g. exact treatment of same-slot service, or the externality cost we omitted), not to a modelling error.

Table 2: Replication of paper Table 1: $\mathbb{E}[W]$ in seconds. “Paper” is the “MDP (cyclic)” column of [1].

ρ	paper	VI ($K = 10$)	VI ($K = 15$)	Δ ($K = 10$)	Δ ($K = 15$)
0.40	4.89	5.061	5.061	+0.171	+0.171
0.60	6.95	6.974	6.974	+0.024	+0.024
0.80	13.5	12.997	13.520	−0.503	+0.020

The $\rho = 0.8$ column is instructive. At $K = 10$, the optimal policy occasionally pushes queue lengths above 10, and our model rejects those arrivals silently — so cars are “lost” and the cost-rate is biased low (-0.503 s vs paper). Lifting the cap to $K = 15$ exceeds the support of the queue distribution under the optimum, and the bias collapses to $+0.020$ s. A complete replication would either fit the externality cost from the paper or push K higher still; for the modest extra computational cost of $K = 15$ (~ 50 s of VI per workload), we consider this match adequate for the homework target.

6.3 PI failure at $\rho = 0.8$

At $\rho = 0.8$, PI’s first improvement step always produces a deterministic policy whose induced Markov chain is *reducible*: starting from certain states, the inactive flows saturate to the buffer cap and never drain, so the chain has multiple recurrent classes (one per starting combination ℓ) each with both inactive flows pinned at K . The Poisson recursion under such a policy has M_n stuck exactly at $2K$ (the per-slot cost in the absorbing class) while m_n creeps upward, so the span never closes. This pathology is *K-independent*: at $K = 10$ we observe $M_n = 20.000$, at $K = 15$ we observe $M_n = 30.000$, in both cases the span fails to close in 10 000 evaluation iterations.

This is the textbook signature of a *weakly* unichain MDP, for which the lecture notes (p. 11) recommend Linear Programming. LP gives the answer at every workload, including $\rho = 0.8$, and agrees with VI to 10^{-5} .

7 Sensitivity discussion

7.1 Effect of the buffer cap K

Table 3 reports $\mathbb{E}[W]^*$ as K varies. At low ρ the queue distribution is concentrated below any reasonable cap and K does not affect the answer (the columns at $\rho = 0.4$ are identical to six decimals). At higher ρ the cap matters: increasing K from 10 to 15 moves the $\rho = 0.8$ result from -0.503 s to $+0.020$ s vs paper.

Table 3: Effect of the buffer cap K on $\mathbb{E}[W]$ (seconds, VI). At low ρ the dependence is negligible; at high ρ the cap truncates the queue distribution.

ρ	$K = 6$	$K = 10$	$K = 15$
0.40	5.061	5.061	5.061
0.60	6.946	6.974	6.974
0.80	10.779	12.997	13.520

7.2 Effect of the workload ρ

Increasing ρ has the obvious primary effect of raising the optimal $\mathbb{E}[W]$ — from 5 s at $\rho = 0.4$ to 13.5 s at $\rho = 0.8$ — but it also changes the *character* of the problem. At low ρ the optimal policy switches frequently (fixed-cycle equivalent 16 s in the paper); at high ρ the optimal policy holds green much longer (cycle 44 s). For our VI this translates into more iterations to converge — ~ 200 at $\rho = 0.4$ versus $\sim 1\,400$ at $\rho = 0.8$, $K = 15$ — more degeneracy in the LP polytope, and as we have seen, the eventual breakdown of PI into multiple recurrent classes.

7.3 Algorithmic suitability

- **VI** is the workhorse: simple, robust, requires only strong aperiodicity (which we obtain by the τ -transformation). It works at every ρ and every K , and was the only solver that produced a clean result for $\rho = 0.8$ at $K = 10$ in our setup.
- **PI** is fast where it works (5–6 outer iterations, sub-second policy evaluations), but its correctness theorem assumes strict unichain. For weakly-unichain MDPs like ours at $\rho = 0.8$, an ad-hoc unichain-respecting initial policy is not enough: improvement can still produce reducible intermediate policies. Larger K does *not* fix the issue.

- **LP** is the most permissive: it works under the weakly-unichain assumption (notes p. 11). Its main practical issues are the degenerate polytope of MDPs (which simplex navigates slowly) and, in our experience, the rank-deficient flow-conservation row that has to be removed by hand for IPM to converge.

8 Conclusion

We replicated the F4C2 row of Haijema and van der Wal’s Table 1 with three independent solvers from the lecture notes. Algorithmic cross-checks pass to 10^{-5} or better at every workload where they apply, and the paper’s mean waiting time per car is reproduced to within 0.2s at every ρ once the buffer cap is taken to be $K = 15$.

The exercise also illustrated, on a textbook-sized problem, two real-world wrinkles in average-cost MDP solving: the *weakly unichain* regime that breaks Policy Iteration, and the *rank-deficient flow-conservation row* that breaks LP interior-point solvers if not removed by hand. Both are predicted by and addressed in the lecture notes.

Reproducibility. The Python code, lecture-notes pseudocode transcription (`docs/algorithms.md`), and machine-readable results (`results/*.json` and `results/*.xlsx`) accompany this report.

References

- [1] R. Haijema and J. van der Wal, An MDP decomposition approach for traffic control at isolated signalized intersections, *Probability in the Engineering and Informational Sciences*, 22(4):587–602, 2008. DOI: 10.1017/S026996480800034X.
- [2] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*, Wiley, 1994.
- [3] DSEFM Lecture Notes on Markov Decision Processes, Spring 2026.